

Project B: Movie Recommendations



Owen Haber, Sam Mard

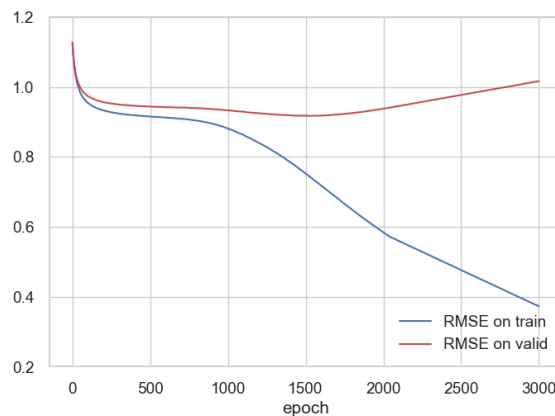
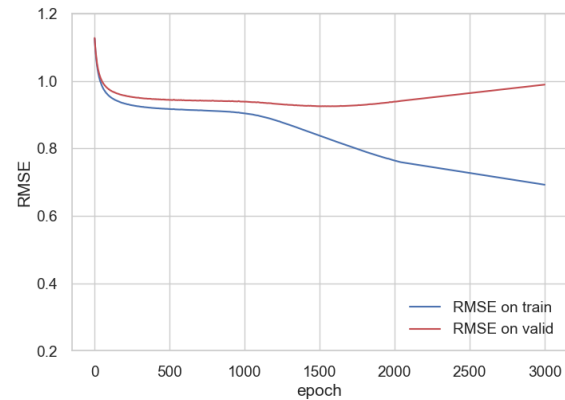
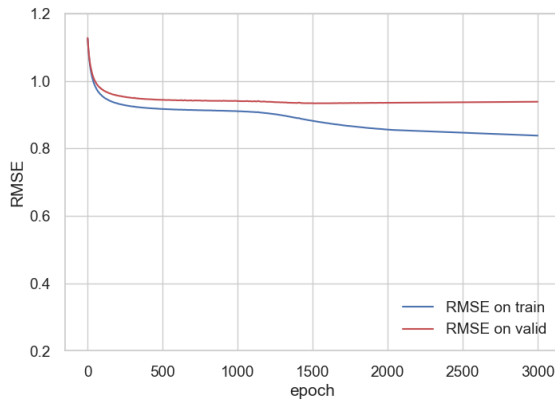
CS 135 Intro To Machine Learning

Prof. Harry Bendekgey

July 20, 2026

Problem 1: One-Vector-Per-Item Collaborative Filtering with SGD and Autograd

Section 1A

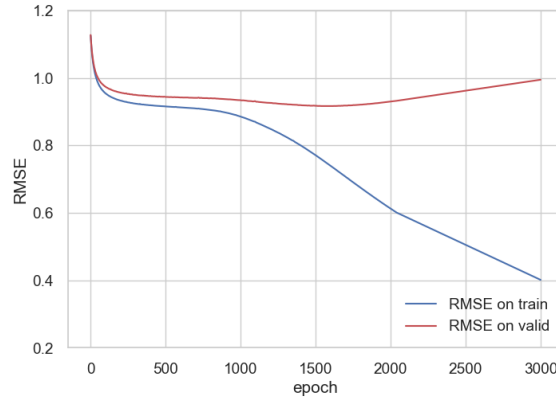


(i) When $K=2$, the model appears to be slightly underfitting, as the training RMSE levels out and the gap between training and validation performance is minimal. As K increases to 10 and 50, the model progressively overfits to the training data as seen by the increasing divergence between training (which continues to drop) and validation (which minimizes then rises) in each subsequent plot; the largest difference between the two happens when $K=50$.

(ii) Increasing K generally improves the 'best' validation set performance. The minimum validation RMSE decreases between $K=2$ and $K=10$, and then again between $K=10$ and $K=50$. This pattern suggests that the higher the number of features, the better the validation performance.

(iii) We picked a step size of 0.7. This was chosen through observation as it provided a good balance between convergence speed and stability. Also, while smaller values resulted in plots that showed no over/underfitting and larger values showed plots that did not converge, a rate of 0.7 allowed us to easily visualize the model's performance in the plots.

Section 1B



RMSE vs. Epoch: $K = 50, \alpha = 0.1$. Minimum Validation RMSE: 0.91646.

(i) We tested three different values: $\alpha = 0.05, 0.1, 0.5$. The best validation RMSE resulted from $\alpha = 0.1$, leading us to pick this value, as both $\alpha = 0.05$ and $\alpha = 0.5$ gave similarly worse performance and slightly different plotting results. Picking this value led to a plot similar to the one for $K=50$ from part 1a, only with the later epochs showing slightly closer errors between training and validation. Compared to 0.1, 0.05 showed more extreme overfitting through larger divergence between errors, while 0.5 showed underfitting similar to the plot for $K=2$ in 1a. Also, the way in which we extracted the optimal errors for each set allowed us to view the test set performance alongside the training and validation set. Interestingly, the test set performed best for $\alpha = 0.5$, however, we ignored this outcome as optimizing on the test set invalidates the model's results and creates bias for that specific testing case.

(ii) To match the same setup and computational efficiency, we used the same step size that we used in part 1a, 0.7.

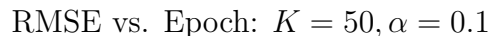
(iii) Yes, we did get better validation set error with this alpha than we did for $K=50$ and $\alpha = 0$. Although the validation RMSE for $\alpha = 0.1$ was only better by 0.00075, it resulted in a noticeably different test RMSE as well.

Section 1C

	Train RMSE	Val RMSE	Test RMSE	Train MAE	Val MAE	Test MAE	Stopping Epoch
$K = 2$	0.87959	0.93359	0.93618	0.69594	0.73291	0.74138	1528
$K = 10$	0.83069	0.92481	0.92821	0.65571	0.72459	0.73306	1544
$K = 50, \alpha = 0$	0.74021	0.91721	0.92421	0.58240	0.71869	0.73162	1536
$K = 50, \alpha = 0.1$	0.74699	0.91646	0.92308	0.58767	0.71789	0.73035	1576

Focusing on RMSE, the best validation performance was from $K = 50$ with $\alpha = 0.1$, which also yields the lowest test RSME. Although the non-regularized $K = 50$ had similar results, the inclusion of L2 regularization resulted in more favorable predictions and yielded a better

Section 1D



3

Problem 2: Open-Ended Recommendation Challenge

Section 2A

For problem 2 we built a recommendation system utilizing the Surprise library, which provides several optimized implementations of collaborative filtering algorithms for recommendation systems. We specifically used the SVD++ and KNNBaseline algorithms, both of which extend the latent factor method we implemented in part 1. Recall the original predictor was

$$\hat{y}_{ij} = \mu + b_i + c_j + \underbrace{\mathbf{u}_i^\top \mathbf{v}_j}_{\text{Latent interaction between user } i \text{ and item } j}$$

which captures user preferences through matrix factorization with per-user and item bias. SVD++ augments this by incorporating "implicit feedback" which allows each user embedding to depend on all items they have interacted with through a new vector term.

$$\hat{r}_{ui} = \mu + b_u + b_i + \underbrace{\mathbf{q}_i^\top \left(\mathbf{p}_u + \frac{1}{\sqrt{|I_u|}} \sum_{j \in I_u} \mathbf{y}_j \right)}_{\text{How well movie } i \text{ matches what user } u \text{ tends to watch or rate}}$$

In contrast, KNNBaseline employs a neighborhood based approach that makes predictions based on similarities between items or users. This allows the model to identify stronger local relationships such as frequently co-watched items and genre clusters, relationships that latent factor models often smooth over.

$$\hat{r}_{ui} = b_{ui} + \underbrace{\frac{\sum_{v \in N_i^{(k)}(u)} \text{sim}(u, v) (r_{vi} - b_{vi})}{\sum_{v \in N_i^{(k)}(u)} \text{sim}(u, v)}}_{\text{Influence of similar neighbors' ratings after adjusting for their biases}}$$

Because SVD++ and KNNBaseline have fundamentally different prediction methods (global dimensions vs. local similarity clusters), they were very strong candidates to form an ensemble. Note that both above equations came from the official scikit-surprise

Following the practices for hyperparameter selection we learned in class, we applied a hyperparameter grid search for both models before combining them in an ensemble. For SVD++, we tuned the latent dimensionality (20, 40, 60, 80), the regularization strength (0.002, 0.005, 0.01, 0.02), the learning rate (0.003, 0.005), and the number of training epochs (30, 50). For KNNBaseline, we searched over several neighborhood sizes (10, 15, 20, 40, 80), similarity metrics (pearson, pearson_baseline), model orientation (item-based or user-based), and baseline regularization strengths for user and item effects (5, 10, 15, 20). We plotted some select parameters against the held-out MAE, and refined our grid searches to ranges that didn't risk underfitting or overfitting. After picking the strongest configuration for each algorithm, we generated the final predictions on the entire training set for each, and combined them

in an ensemble, using a weighted average of their individual outputs. We chose to weight SVD++ at 0.6 and KNNBaseline at 0.4, which out of a manual sweep of weight ratios yielded the best ensemble error on the dev test split (validation split).

Section 2B

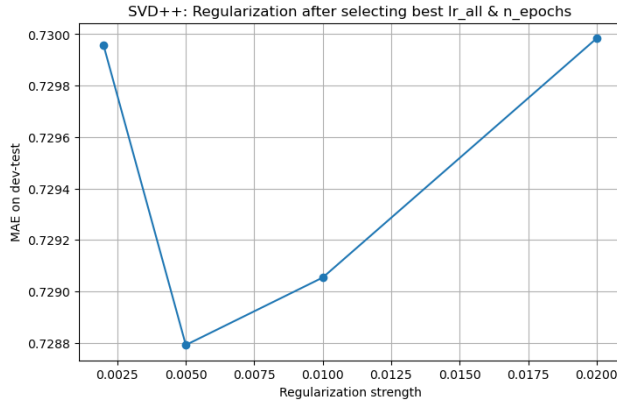


Figure 2: SVD++ Regularization

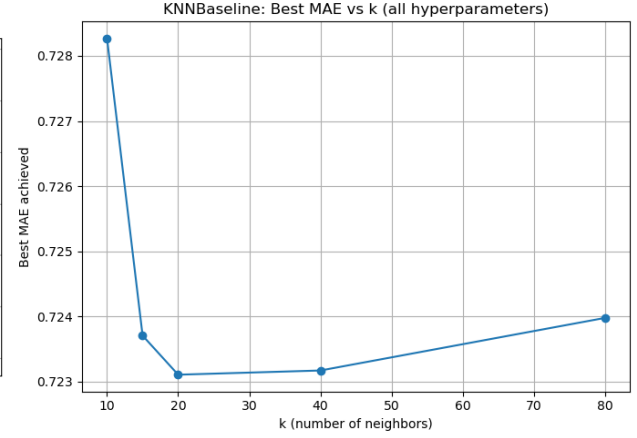


Figure 3: KNNBaseline Neighborhood Size

The rightmost plot depicts the plot of the neighborhood size k against validation MSE. The plot shows small k values overfit by relying on too few neighbors, and larger k under-fit by over-smoothing the signals we get from local relationships. The curve reveals a clear performance minimum at around $k = 20$, showing the need to balance model complexity to avoid extreme behavior.

The leftmost plot depicts how regularization strength affects the performance of SVD++. Too little regularization leads to slight overfitting, while too much causes the model to under-fit. The best held-out MAE occurs at around $\lambda = 0.005$, a moderate value range that confirms the fact that proper regularization is necessary for good generalization.

Section 2C

Method	Validation MAE	MAE (leaderboard)
LF (Problem 1, $K = 50$, $\alpha = 0.1$)	0.71789	N/A
SVD++ (best)	0.722594	N/A
KNNBaseline (best)	0.723105	N/A
Ensemble (SVD++ + KNN) (chosen)	0.713930	0.701

The latent factorization on its own from problem one outperforms both SVD++ and KNNBaseline on their own. This highlights the strength of item-based and user-based latent interaction

as opposed to strictly neighborhood and global ratings that the surprise algorithms rely on. When combined in an ensemble however, the ensemble performs slightly better than the algorithm in part one, capturing both global latent preference structure and local similarity signals, which is not possible for the latent factorization.

The ensemble performed extremely well on the leader board test set, earning a MAE of 0.701. This shows a good ability to generalize to new data, as it performed better on the unseen test data than the validation split from training. This means it didn't overfit to our validation split, and showcases an ability to capture stable patterns within the movielense data set.

Section 2D

Our final solution combines both the SVD++ and KNNBaseline in an ensemble, allowing us to capture both global latent patterns and neighborhood signals via item similarity. This makes the approach very effective for standard explicit rating datasets like MovieLens, where users have somewhat stable preferences and have rated enough items for both latent factors and neighborhoods pattern to be learned. This method has some clear limitations. Both SVD++ and KNNBaseline struggle with sparse data, such when users don't repeat behavior, or recommendations are based on outside context or additional metadata. The method does not use any item or user metadata, so it is not suitable for problems where side information is necessary to fill in missing structure or combat sparsity.

If we had more time, our focus would be on optimizing the ensemble strategy to ensure maximum performance from combined models. The current ensemble average (0.6 SVD++ and 0.4 KNNBaseline) that was selected by a manual sweep over different weights, would be replaced by a higher fidelity Linear Regression model to learn the exact optimal weights by minimizing combined error on the dev test set. Here, we could further expand complexity by introducing at least one more diverse algorithm such as NMF or Slope One, both algorithms offered in the sci-kit learn surprise library, to better cover different types of error. We would then use our regression model to determine the exact contribution of each model to our final prediction, resulting in a more robust and better generalized weighting system.

Looking at both parts of this project, there were multiple key takeaways surrounding how recommender systems are constructed. The first takeaway surrounds hyperparameter tuning, specifically L2 regularization α , when configuring our model in part 1. We confirmed that increasing the number of factors, K , immediately improved validation accuracy despite risking overfitting. Adding the regularization penalty helped decrease that risk by lowering the discrepancies between training and validation errors. However, this number must be fine tuned, as lower penalties seem to cause overfitting while larger ones may cause underfitting. To optimize performance, a range of values must be observed.

Our second insight is about the interpretation of latent embeddings. The results from the scatterplot in 1d revealed interesting visualizations on how the computer clusters the data. Although the semantics are unclear, the model clustered some movies by their user ratings clearly by franchise, genre, or director. More profoundly, there is a clear aspect of audience perception that is involved with the clustering, as seen by how lower rated sequels are placed on the plot relative to their originals. These patterns suggest that the computer successfully filtered data by different objective facts, as well as perceived quality.

Once we got to part 2, it was interesting to find that all of the individual algorithms had theoretical performance limits of around 0.72 - 0.75 MAE on the 10k MovieLens data set, as reported by the scikit surprise package website. Completely redesigning our method using different surprise algorithms and ensembling only led to a marginal increase in performance, and we couldn't break the 0.700 barrier on the leaderboard even though we were very close. It makes us wonder if simply having more data, such as the 1M MovieLens data set would increase our test performance, or maybe a more complex ensemble. Overall, the experience highlighted how model improvements eventually become constrained by dataset size and inherent noise, and how performance plateaus naturally signal the need for a different modeling approach.